

The Linear Model (LM)

T. Florian Jaeger

September 2, 2026

Readings and assignments in preparation of this class

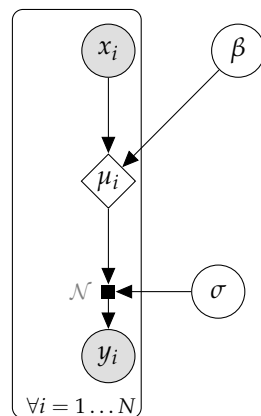
Optionally: Before the next meeting, you might find it helpful to read and *work through* this document. Please note that this document is providing R code only. Some of the steps described here might have less direct solutions in Matlab or Python, so it is recommended that you start early.

About this document

This PDF is generated from an R markdown document (using the `tufte` package for styling). If you're interested in understanding the code that generated the examples presented below, you can clone the repo. Both the PDF and the R markdown that the PDF is generated from are in the `scripts/` folder. You can go through the document line by line, or make changes and 'knit' a new PDF with the press of a button.

The linear model (LM)

Recall the linear model:



and the specific LM we fit to a single participant from Tan & Jaeger (2025):

```
Call:
lm(formula = Response.RT ~ 1 + VOT, data = dd)
```

Figure 1: A graphical model visualization of the linear model. Shaded circles are observable variables. White circles are latent variables. These are the variables that constitute the degrees of freedom of the model, the parameters that are fit to the data. Diamonds shapes indicate variables that are determined by other variables. The index $i = 1, \dots, N$ refers identifies the observations (usually, the rows) in your data. Note that σ , unlike μ_i is assumed to be constant across observations. This is the assumption of the homogeneity of variance.

```

Residuals:
    Min       1Q   Median       3Q      Max
-749.9 -451.7 -287.1    2.4 3685.0

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 2205.812    191.953   11.491  <2e-16 ***
VOT          -7.595      4.556   -1.667    0.1 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 856.6 on 70 degrees of freedom
Multiple R-squared:  0.03818,    Adjusted R-squared:  0.02444
F-statistic: 2.779 on 1 and 70 DF,  p-value: 0.1

```

In this document, we'll go through an example of how this model was fit. We'll start with the perhaps most intuitive fitting algorithm, the *least-square fitting* algorithm. We'll then show how this is equivalent to a more general approach, *likelihood maximization*, which can be used to fit many other types of models.

Least-square fitting

How did the model fitting algorithm arrive at the estimates and what do they present? What do the standard errors (SEs) of the estimates represent? Intuitively, we know that the estimates are the “best-fitting” estimates for the coefficients. What this means depends on how the goodness of fit is calculated.

One way to find the best-fitting estimates for the coefficients of an LM is to minimize the total prediction error. For each observation in our data, the prediction error or *residual* of the model m is defined as the difference between the predicted value of y , $E_m[y]$, and the actually observed value of y , i.e., $residual = y - E_m[y]$. Since we don't want negative and positive prediction errors (resulting from over- and under-predicting y , respectively) to cancel each other out, we cannot simply sum those prediction errors to get an estimate of how well the model fits our data. Instead, we sum the squared residuals, giving us the residual sum of squares (RSS).¹ Finding the best-fitting estimates then becomes the identical to finding the estimates that minimize the RSS.

To better understand what happens when we find the best-fitting estimates by minimizing the RSS, let's define a few function that help use deliver predictions just like the final LM presented above does, but for arbitrary intercept values a and arbitrary slope values b for the effect of VOT. We also define a function that gets the prediction error for each observation in our data, and a function that sums up the squared prediction error:

¹ One could instead use any other function that would result in only positive prediction errors (e.g., taking the *absolute* prediction error). However, using the squared residuals has both appealing properties, and it turns out to be identical—for LMs—to maximizing the likelihood of the data.

```

get_prediction <- function(a, b, data = dd) {
  a + b * dd$VOT
}

get_prediction_error <- function(a, b, data = dd) {
  dd$Response.RT - get_prediction(a, b, data = data)
}

get_RSS <- function(a, b, data = dd) {
  sum(get_prediction_error(a, b, data = data)^2)
}

```

Now we can use these functions to visualize the predictions and calculate the RSSs for different hypotheses about a and b . You can use Acrobat Viewer to animate through the following examples (PDF system preview typically won't work):

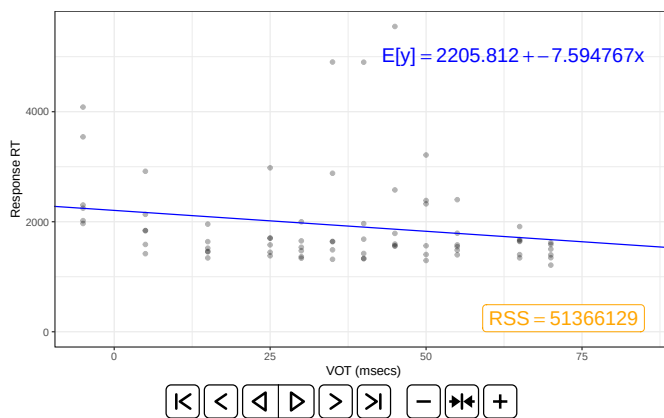


Figure 2: Raw RTs for one participant across the six test blocks (points), along with prediction line for different intercepts a and slopes b for the model $RT \sim 1 + VOT$. Compare how the RSS changes, depending on a and b .

We can also visualize the RSS—i.e., the function we're trying to minimize—depending on a wide range of values for a and b . This illustrates two things. First, the estimates provided by the LM at the start of this section indeed minimize the RSS of the model. Second, RSS are a convex function of the parameters in the model (here a and b), with a single optimum. This very appealing property of LMs allows for efficient fitting algorithms that are *guaranteed* to converge within an arbitrary finite number of steps against the optimal parameters. Basically, it means that going downhill is bound to get you closer to the optimum, rather than stuck in a local minimum (as can be the case for more complex optimization problems, e.g., for many types of non-linear models).

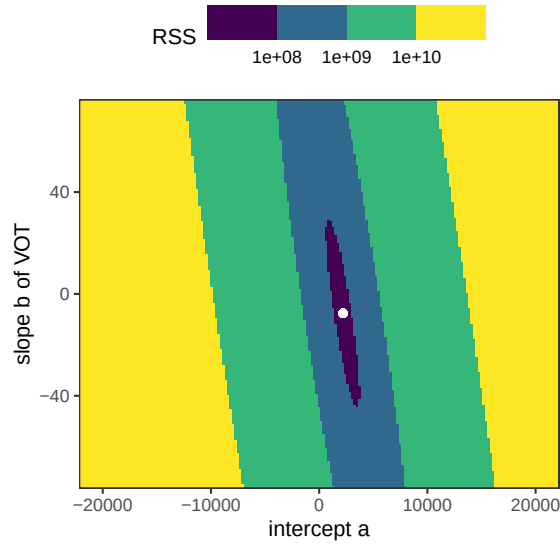
Here's an example of how one can use a general optimizer to find the best-fitting estimates for an LM. The example uses R's `optim()` function but you can try to implement it in your preferred programming language.² You can compare its output to the estimates reported at the start of the section:

```

# let's store the optimizer's steps in a data.frame
d.trace <- tibble(.rows = 0)

```

² By default, `optim()` uses Quasi-Newtonian optimization. This works just fine for the present purpose.



```
o <-
optim(
  # Starting values for a and b
  par = c(0, 0),
  fn = function(x) {
    rss <- get_RSS(x[1], x[2]);
    # Uncomment this line to see how optim() searches the parameter space
    # message(paste0("a = ", x[1], ", b = ", x[2], "; RSS = ", rss, "\n"))
    # <- assigns values to *global* variables (outside the function)
    d.trace <- d.trace %>% bind_rows(tibble(a = x[1], b = x[2], RSS = rss))
    return(rss)
  })

# print optimal parameters found by optim()
print(o$par)

[1] 2205.921965   -7.594757
```

Want to challenge yourself? Apply what you learned

While minimization of RSS works well for LMs, there is a more generally applicable approach that yields the same results for LMs but also can be used for the fitting of many others types of models, *likelihood maximization*. Under this approach, we find the best-fitting estimates that maximize the likelihood of the observed outcomes y . Recall that the LM assumes y to be normally distributed around the predicted $E[y]$. This means we can calculate the *likelihood* of each observation under the hypothesized normal distribution. This normal distribution is a function of the hypothesized values for all the model's coefficients (a and b in the example above) and the value hypothesized for σ . For any hypothesis about a , b , and σ , we can get the density of an observation y under $\mathcal{N}(a + bx, \sigma)$. By multiplying the densities of all observations, we can get the data likelihood

Figure 3: RSS for the model $RT \sim 1 + VOT$, depending on its intercept a and slope b . The white point shows that location of the estimates given in the LM output at the beginning of this section.

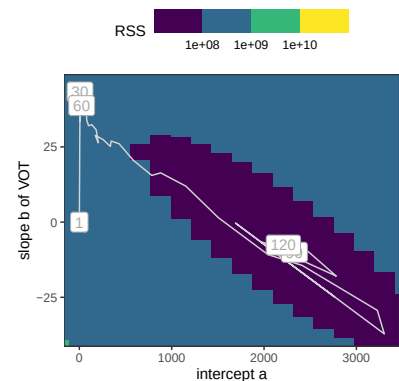


Figure 4: Same as above but zoomed in and showing the path of the optimizer superimposed. Labels show step of optimizer

under the hypothesized values for a , b , and σ . In practice, we instead sum the log-transformed likelihoods. This is numerically more stable, and the x that maximizes $\log(f(x))$ is the same that maximizes $f(x)$.

- Define a function that calculates the sum of log-densities for the model $RT \sim 1 + VOT$ given a set of arguments a , b , and σ .³
- Make a figure like the one shown above for RSS. For this figure set σ to the σ from the LM at the start of this section.⁴ We don't expect the likelihood values to be identical to the RSS values—they are on completely different scales. But do the surfaces have similar shapes (curvatures)?
- To further understand the relationship between OLS and ML fitting of linear regression with normally distributed residuals, see [this presentation, slides 99-122](https://faculty.washington.edu/cadolph/mle/topic2.p.pdf) (it's a lot of slides but that's just because each derivation step is presented as a separate slide).

³ In R, `dnorm()` will give you the density of an observation or vector of observations y under a Normal with the specified mean and standard deviations. Use the optional argument `log = TRUE` to get log densities.

⁴ In R, use `sigma()` to extract σ from an LM.

Session info

```
devtools::session_info()
```

```
- Session info -----
setting  value
version  R version 4.5.2 (2025-10-31)
os       macOS Sonoma 14.5
system   aarch64, darwin20
ui       X11
language (EN)
collate  en_US.UTF-8
ctype    en_US.UTF-8
tz       America/New_York
date     2026-09-02
pandoc   3.10 @ /opt/homebrew/bin/ (via rmarkdown)
quarto   1.9.38 @ /usr/local/bin/quarto

- Packages -----
package      * version  date (UTC) lib source
backports    1.5.1    2026-04-03 [1] CRAN (R 4.5.2)
broom        * 1.0.13   2026-05-14 [1] CRAN (R 4.5.2)
cachem       1.1.0    2024-05-16 [1] CRAN (R 4.5.0)
cli          3.6.6    2026-04-09 [1] CRAN (R 4.5.2)
crayon       1.5.3    2024-06-20 [1] CRAN (R 4.5.0)
devtools     2.5.2    2026-04-30 [1] CRAN (R 4.5.2)
digest       0.6.39   2025-11-19 [1] CRAN (R 4.5.2)
dplyr        * 1.2.1    2026-04-03 [1] CRAN (R 4.5.2)
ellipsis     0.3.3    2026-04-04 [1] CRAN (R 4.5.2)
evaluate     1.0.5    2025-08-27 [1] CRAN (R 4.5.0)
farver       2.1.2    2024-05-13 [1] CRAN (R 4.5.0)
fastmap      1.2.0    2024-05-15 [1] CRAN (R 4.5.0)
forcats      * 1.0.1    2025-09-25 [1] CRAN (R 4.5.0)
fs           2.1.0    2026-04-18 [1] CRAN (R 4.5.2)
generics     0.1.4    2025-05-09 [1] CRAN (R 4.5.0)
gganimate    * 1.0.11   2025-09-04 [1] CRAN (R 4.5.0)
```

ggplot2	* 4.0.3	2026-04-22	[1]	CRAN	(R 4.5.2)
ggthemes	* 5.2.0	2025-11-30	[1]	CRAN	(R 4.5.2)
gifski	1.32.0-2	2025-03-18	[1]	CRAN	(R 4.5.0)
glue	1.8.1	2026-04-17	[1]	CRAN	(R 4.5.2)
gtable	0.3.6	2024-10-25	[1]	CRAN	(R 4.5.0)
hms	1.1.4	2025-10-17	[1]	CRAN	(R 4.5.0)
htmltools	0.5.9	2025-12-04	[1]	CRAN	(R 4.5.2)
knitr	* 1.51	2025-12-20	[1]	CRAN	(R 4.5.2)
labeling	0.4.3	2023-08-29	[1]	CRAN	(R 4.5.0)
lifecycle	1.0.5	2026-01-08	[1]	CRAN	(R 4.5.2)
lubridate	* 1.9.5	2026-02-04	[1]	CRAN	(R 4.5.2)
magrittr	* 2.0.5	2026-04-04	[1]	CRAN	(R 4.5.2)
MASS	* 7.3-66	2026-07-15	[1]	CRAN	(R 4.5.2)
memoise	2.0.1	2021-11-26	[1]	CRAN	(R 4.5.0)
otel	0.2.0	2025-08-29	[1]	CRAN	(R 4.5.0)
pillar	1.11.1	2025-09-17	[1]	CRAN	(R 4.5.0)
pkgbuild	1.4.8	2025-05-26	[1]	CRAN	(R 4.5.0)
pkgconfig	2.0.3	2019-09-22	[1]	CRAN	(R 4.5.0)
pkgload	1.5.3	2026-06-15	[1]	CRAN	(R 4.5.2)
prettyunits	1.2.0	2023-09-24	[1]	CRAN	(R 4.5.0)
progress	1.2.3	2023-12-06	[1]	CRAN	(R 4.5.0)
progressr	1.0.0	2026-07-04	[1]	CRAN	(R 4.5.2)
purrr	* 1.2.2	2026-04-10	[1]	CRAN	(R 4.5.2)
R6	2.6.1	2025-02-15	[1]	CRAN	(R 4.5.0)
RColorBrewer	1.1-3	2022-04-03	[1]	CRAN	(R 4.5.0)
readr	* 2.2.0	2026-02-19	[1]	CRAN	(R 4.5.2)
rlang	1.3.0	2026-07-05	[1]	CRAN	(R 4.5.2)
rmarkdown	2.31	2026-03-26	[1]	CRAN	(R 4.5.2)
rstudioapi	0.19.0	2026-06-11	[1]	CRAN	(R 4.5.2)
S7	0.2.2	2026-04-22	[1]	CRAN	(R 4.5.2)
scales	1.4.0	2025-04-24	[1]	CRAN	(R 4.5.0)
sessioninfo	1.2.4	2026-06-04	[1]	CRAN	(R 4.5.2)
stringi	1.8.9	2026-08-04	[1]	CRAN	(R 4.5.2)
stringr	* 1.6.0	2025-11-04	[1]	CRAN	(R 4.5.0)
tibble	* 3.3.1	2026-01-11	[1]	CRAN	(R 4.5.2)
tidyr	* 1.3.2	2025-12-19	[1]	CRAN	(R 4.5.2)
tidyselect	1.2.1	2024-03-11	[1]	CRAN	(R 4.5.0)
tidyverse	* 2.0.0	2023-02-22	[1]	CRAN	(R 4.5.0)
timechange	0.4.0	2026-01-29	[1]	CRAN	(R 4.5.2)
tinytex	0.60	2026-06-16	[1]	CRAN	(R 4.5.2)
tufte	* 0.15.0	2026-05-21	[1]	CRAN	(R 4.5.2)
tweenr	2.0.3	2024-02-26	[1]	CRAN	(R 4.5.0)
tzdb	0.5.0	2025-03-15	[1]	CRAN	(R 4.5.0)
usethis	3.2.1	2025-09-06	[1]	CRAN	(R 4.5.0)
vctrs	0.7.3	2026-04-11	[1]	CRAN	(R 4.5.2)
viridisLite	0.4.3	2026-02-04	[1]	CRAN	(R 4.5.2)
withr	3.0.3	2026-06-19	[1]	CRAN	(R 4.5.2)
xfun	0.60	2026-07-09	[1]	CRAN	(R 4.5.2)
yaml	2.3.12	2025-12-10	[1]	CRAN	(R 4.5.2)

[1] /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library

* -- Packages attached to the search path.
